

BÀI TẬP THỰC HÀNH: Xây dựng Web API với ASP.NET Core (.NET 10) theo mô hình Layer, DI, DTO và LINQ Join

1. Mục tiêu bài tập

Xây dựng một **ASP.NET Core Web API** bằng **.NET 10 (phiên bản mới nhất)** với các yêu cầu sau:

- Chia project theo **layer rõ ràng**.
- Sử dụng **Service Layer** và **Dependency Injection (DI)**.
- Không dùng database, chỉ dùng **seed data bằng collection list**.
- Bắt buộc dùng **DTO** thay vì trả trực tiếp Entity.
- Thực hiện **Join giữa 2 collection list**:
 - Join bằng **Method Syntax**
 - Join bằng **Query Syntax**
- Tìm hiểu và áp dụng:
 - **Exception pattern cơ bản**
 - **Chuẩn hoá API response**
 - **Left Join** để xử lý trường hợp dữ liệu không khớp
- Controller phải **mỏng**, mọi xử lý nghiệp vụ phải nằm trong Service.

Lưu ý bắt buộc: phải dùng **.NET 10**. Không chấp nhận bài làm dùng version cũ.

2. Bối cảnh bài toán

Xây dựng Web API quản lý **vật tư xây dựng** và **nhà cung cấp**.

2.1. Thực thể dữ liệu

Material

- Id
- Name
- Unit
- UnitPrice
- SupplierId (cho phép null)

Supplier

- Id
- Name
- Address
- ContactPhone

2.2. Ý nghĩa nghiệp vụ

- Mỗi vật tư có thể thuộc một nhà cung cấp.
 - Có vật tư **chưa có nhà cung cấp**.
 - Khi trả dữ liệu ra API, không trả trực tiếp entity gốc mà phải map sang DTO.
 - API cần hiển thị được thông tin nhà cung cấp tương ứng cho vật tư.
-

3. Yêu cầu kiến thức bắt buộc

Sinh viên phải thể hiện được các nội dung sau trong bài làm:

- OOP cơ bản
 - Interface
 - Service Layer
 - Dependency Injection
 - DTO
 - LINQ Method Syntax
 - LINQ Query Syntax
 - Join giữa collection list
 - Exception pattern cơ bản
 - Chuẩn hoá API response
-

4. DTO là gì? Vì sao bài này bắt buộc dùng DTO?

4.1. Khái niệm DTO

DTO (Data Transfer Object) là object dùng để **truyền dữ liệu ra/vào API**, thay vì dùng trực tiếp entity nội bộ.

4.2. Vì sao không trả thẳng Entity?

Nếu trả trực tiếp entity **Material**, API có thể:

- Lộ ra các trường không cần thiết, ví dụ **SupplierId**
- Trả dữ liệu chưa đúng nhu cầu hiển thị
- Làm code khó kiểm soát khi yêu cầu thay đổi

4.3. Trong bài này, DTO dùng để làm gì?

Ví dụ entity **Material** có **SupplierId**, nhưng phía client cần:

- **SupplierName**
- **SupplierAddress**

Vì vậy sinh viên phải dùng **Select** để map dữ liệu từ join result sang DTO.

4.4. Yêu cầu bắt buộc về DTO

API **không được** trả:

- **SupplierId**

- Toàn bộ object **Supplier**
- Entity gốc chưa qua xử lý

API **phải trả** DTO có ý nghĩa hiển thị.

5. Cấu trúc thư mục gợi ý

Sinh viên có thể tự tổ chức, nhưng khuyến khích theo cấu trúc sau:

```
ConstructionMaterialsApi/  
├── Controllers/  
│   └── MaterialsController.cs  
├── Models/  
│   ├── Entities/  
│   │   ├── Material.cs  
│   │   └── Supplier.cs  
│   ├── Dtos/  
│   │   ├── MaterialDto.cs  
│   │   └── MaterialDetailDto.cs  
│   └── Common/  
│       └── ApiResponse.cs  
├── Services/  
│   ├── Interfaces/  
│   │   └── IMaterialService.cs  
│   └── Implementations/  
│       ├── MethodSyntaxMaterialService.cs  
│       └── QuerySyntaxMaterialService.cs  
├── Data/  
│   └── SeedData.cs  
├── Exceptions/  
│   └── UserFriendlyException.cs  
└── Program.cs
```

Có thể thay đổi tên thư mục, nhưng phải giữ rõ tư duy phân lớp.

6. Seed data mẫu

Sinh viên được phép copy seed data dưới đây để thực hành.

6.1. Supplier seed data

```
public static List<Supplier> Suppliers => new List<Supplier>
{
    new Supplier { Id = 1, Name = "Thép Hòa Phát", Address = "Hà Nội",
ContactPhone = "0901000001" },
    new Supplier { Id = 2, Name = "Xi măng Xuân Thành", Address = "Hà Nam",
ContactPhone = "0901000002" },
    new Supplier { Id = 3, Name = "Gạch Viglacera", Address = "Bắc Ninh",
ContactPhone = "0901000003" },
    new Supplier { Id = 4, Name = "Cát sạch Sông Đà", Address = "Hòa Bình",
ContactPhone = "0901000004" }
};
```

6.2. Material seed data

```
public static List<Material> Materials => new List<Material>
{
    new Material { Id = 1, Name = "Xi măng PC40", Unit = "Tấn", UnitPrice =
1500000, SupplierId = 2 },
    new Material { Id = 2, Name = "Thép cuộn Ø6", Unit = "Tấn", UnitPrice =
16500000, SupplierId = 1 },
    new Material { Id = 3, Name = "Thép thanh Ø18", Unit = "Tấn", UnitPrice =
17200000, SupplierId = 1 },
    new Material { Id = 4, Name = "Gạch đỏ đặc", Unit = "Viên", UnitPrice = 1200,
SupplierId = 3 },
    new Material { Id = 5, Name = "Cát vàng", Unit = "Khối", UnitPrice = 350000,
SupplierId = 4 },
    new Material { Id = 6, Name = "Đá 1x2", Unit = "Khối", UnitPrice = 420000,
SupplierId = 4 },
    new Material { Id = 7, Name = "Gạch block", Unit = "Viên", UnitPrice = 9500,
SupplierId = 3 },
    new Material { Id = 8, Name = "Xi măng PCB30", Unit = "Tấn", UnitPrice =
1400000, SupplierId = 2 },
    new Material { Id = 9, Name = "Ống nhựa PVC", Unit = "Mét", UnitPrice = 67000,
SupplierId = null },
    new Material { Id = 10, Name = "Sơn chống thấm", Unit = "Thùng", UnitPrice =
890000, SupplierId = null }
};
```

Trong dữ liệu mẫu có **vật tư không có nhà cung cấp**. Đây là dữ liệu cố ý để kiểm tra việc hiểu **Inner Join** và **Left Join**.

7. Yêu cầu interface và service

7.1. Interface chung

Sinh viên tạo interface:

```
public interface IMaterialService
{
    string ImplementationName { get; }
    IEnumerable<MaterialDto> GetAll();
    MaterialDetailDto GetById(int id);
}
```

7.2. Hai implementation bắt buộc

Cách 1: Method Syntax

Tạo class:

- `MethodSyntaxMaterialService`

Toàn bộ LINQ phải viết theo kiểu:

- `.Join()`
- `.Where()`
- `.Select()`
- `.FirstOrDefault()`
- `.GroupJoin()` nếu sinh viên nghiên cứu thêm cho left join

Cách 2: Query Syntax

Tạo class:

- `QuerySyntaxMaterialService`

Toàn bộ LINQ phải viết theo kiểu:

- `from`
- `join`
- `on`
- `equals`
- `where`
- `select`

Không được viết lẫn lộn lung tung. Mỗi service phải thể hiện rõ đúng style của nó.

8. Yêu cầu về DI

Sinh viên phải đăng ký **1 trong 2 implementation** vào `IMaterialService` trong `Program.cs`.

Ví dụ:

- Đăng ký `MethodSyntaxMaterialService`
- Hoặc đăng ký `QuerySyntaxMaterialService`

Mục tiêu:

- Sinh viên phải biết đổi implementation thông qua DI
- Hiểu rằng Controller không phụ thuộc trực tiếp class cụ thể, chỉ phụ thuộc interface

9. Kiểm tra rẽ nhánh DI bằng ImplementationName

9.1. Yêu cầu

Trong JSON trả về của API, luôn phải có trường:

- `sourceImplementation`

Giá trị lấy từ property:

- `ImplementationName`

Ví dụ:

- `"Method Syntax Style"`
- `"Query Syntax Style"`

9.2. Mục đích kiểm tra

Nếu sinh viên đăng ký service kiểu Method Syntax nhưng kết quả trả về lại ghi `"Query Syntax Style"` thì chứng tỏ:

- Chưa hiểu DI
- Hoặc copy code nhưng không sửa đúng logic

10. Yêu cầu API tối thiểu

Sinh viên phải xây dựng ít nhất các API sau:

10.1. GET /api/materials

Trả danh sách vật tư sau khi join với nhà cung cấp.

10.2. GET /api/materials/{id}

Trả chi tiết 1 vật tư theo id.

10.3. GET /api/materials/inner-join

Trả kết quả dùng **inner join**.

10.4. GET /api/materials/left-join

Trả kết quả dùng **left join**.

Endpoint `left-join` là phần sinh viên phải tự tìm hiểu thêm.

11. Yêu cầu DTO đầu ra

11.1. DTO danh sách

Gợi ý các trường nên có:

- Id
- Name
- Unit
- UnitPrice
- SupplierName
- SupplierAddress
- SourceImplementation

11.2. DTO chi tiết

Có thể gồm:

- Id
- Name
- Unit
- UnitPrice
- SupplierName
- SupplierAddress
- SourceImplementation

11.3. Không được trả ra

- `SupplierId`
- Toàn bộ object `Supplier`
- Entity chưa map DTO

Bắt buộc dùng `Select` để map kết quả sang DTO.

12. Yêu cầu chuẩn hoá API response

Tất cả API phải trả về theo mẫu chuẩn hoá sau:

```
public class ApiResponse<T>
{
    public bool Success { get; set; }
    public string Message { get; set; }
    public T Data { get; set; }
    public List<string> Errors { get; set; }

    public static ApiResponse<T> SuccessResponse(T data, string message =
"Success")
    {
```

```
        return new ApiResponse<T>
        {
            Success = true,
            Message = message,
            Data = data,
            Errors = null
        };
    }

    public static ApiResponse<T> FailureResponse(string message, List<string>
errors = null)
    {
        return new ApiResponse<T>
        {
            Success = false,
            Message = message,
            Data = default,
            Errors = errors
        };
    }
}
```

12.1. Ví dụ response thành công

```
{
  "success": true,
  "message": "Success",
  "data": {
    "id": 1,
    "name": "Xi măng PC40",
    "supplierName": "Xi măng Xuân Thành",
    "unitPrice": 1500000
  },
  "errors": null
}
```

12.2. Ví dụ response thất bại

```
{
  "success": false,
  "message": "Vật tư với ID 99 không tồn tại trong hệ thống.",
  "data": null,
  "errors": null
}
```

13. Yêu cầu về exception pattern

Sinh viên phải tìm hiểu và áp dụng **exception pattern cơ bản**.

13.1. Tình huống bắt buộc xử lý

Khi gọi:

- `GET /api/materials/99`

Nếu không tồn tại id 99 thì:

- Không trả dữ liệu rỗng một cách im lặng
- Phải xử lý lỗi có chủ đích

13.2. Hướng triển khai gợi ý

Sinh viên có thể tự nghiên cứu các từ khoá:

- `custom exception asp.net core`
- `NotFoundException`
- `global exception handling asp.net core`
- `middleware exception handling asp.net core`
- `exception pattern service layer`

13.3. Mục tiêu

Sinh viên cần hiểu:

- Nơi nào nên throw exception
- Nơi nào nên bắt lỗi
- Cách biến exception thành response chuẩn hoá

14. Yêu cầu về Clean Service Layer

14.1. Quy định bắt buộc

Controller chỉ nên làm các việc:

- Nhận request
- Gọi service
- Trả response chuẩn hoá

14.2. Không được làm trong Controller

- Viết join
- Viết where phức tạp
- Viết select map DTO
- Tính toán logic nghiệp vụ

14.3. Mục tiêu chấm

Nếu Controller chứa quá nhiều LINQ thì xem như:

- Chưa hiểu Service Layer
- Chưa tách trách nhiệm đúng

Controller chỉ được gọi kiểu: `_service.GetAll()` hoặc `_service.GetById(id)`

15. Kiểm tra bản chất của Inner Join

15.1. Yêu cầu

Sinh viên dùng **inner join** giữa **Material** và **Supplier**.

15.2. Điều cần quan sát

Danh sách gốc có 10 vật tư, nhưng khi inner join có thể không còn đủ 10 phần tử, vì:

- Những vật tư không có **SupplierId**
- Hoặc không join được supplier tương ứng
- Sẽ bị loại khỏi kết quả

15.3. Bài kiểm tra hiểu bản chất

Sinh viên phải tự trả lời bằng **text viết tay** trong phần nộp bài:

"Tại sao kết quả API lại thiếu 1 hoặc nhiều vật tư so với danh sách mẫu?"

Không dùng AI để viết phần giải thích này.

15.4. Ý đúng cần hướng đến

- Do **Inner Join** chỉ lấy các bản ghi có dữ liệu khớp ở cả hai phía

16. Yêu cầu nâng cao: Left Join

16.1. Mục tiêu

Sinh viên phải tự tìm hiểu cách thực hiện **left join với collection list** để vẫn lấy được cả vật tư không có nhà cung cấp.

16.2. Bắt buộc tự nghiên cứu

Không cho code mẫu hoàn chỉnh phần này.

16.3. Từ khoá gợi ý để tự tìm hiểu

- `linq left join method syntax`
- `linq left join query syntax`
- `GroupJoin DefaultIfEmpty`
- `left join in memory collection c#`
- `linq query syntax into DefaultIfEmpty`

16.4. Kết quả mong đợi

Các vật tư như:

- Ống nhựa PVC
- Sơn chống thấm

vẫn phải xuất hiện trong danh sách `left-join`, dù không có nhà cung cấp.

16.5. Gợi ý xử lý DTO khi không có supplier

Sinh viên tự quyết định cách hiển thị, ví dụ:

- `SupplierName = "Chưa có nhà cung cấp"`
- `SupplierAddress = null`
- Hoặc cách khác hợp lý

17. Từ khoá nghiên cứu bắt buộc

Sinh viên phải chủ động tìm hiểu và áp dụng từ các từ khoá sau:

- `asp net core web api .net 10`
- `dependency injection asp net core`
- `service layer asp net core`
- `dto in asp net core web api`
- `linq join method syntax`
- `linq join query syntax`
- `linq left join c#`
- `groupjoin defaultifempty`
- `custom exception asp net core`
- `global exception handling asp net core`
- `standard api response asp net core`
- `swagger test web api asp net core`

18. Yêu cầu đầu ra khi nộp bài

Sinh viên nộp:

- Source code project
- Ảnh chụp Swagger hoặc Postman cho các API
- Ảnh thể hiện rõ trường `sourceImplementation`
- Ảnh kết quả `inner join`
- Ảnh kết quả `left join`
- File text ngắn tự viết trả lời câu hỏi:
 - Vì sao inner join bị thiếu dữ liệu?

19. Tiêu chí chấm điểm

19.1. Cấu trúc project và phân lớp (2 điểm)

- Có Controller, Service, Interface, DTO, SeedData
- Tách lớp rõ ràng

19.2. DTO và map dữ liệu đúng (2 điểm)

- Không trả entity trực tiếp
- Không lộ `SupplierId`
- Có `SupplierName`, `SupplierAddress`

19.3. DI và implementation rõ ràng (2 điểm)

- Đăng ký service đúng
- `sourceImplementation` đúng với service đang dùng

19.4. Inner Join và Query/Method Syntax (2 điểm)

- Có đủ 2 cách viết join
- Hiểu được inner join làm mất dữ liệu không khớp

19.5. Left Join + exception pattern + api response (2 điểm)

- Có endpoint left join
- Có xử lý not found
- Có chuẩn hoá response

20. Yêu cầu chống làm bài hời hợt

Bài làm sẽ bị đánh giá chưa đạt nếu có một trong các dấu hiệu sau:

- Dùng .NET version cũ
- Không dùng DTO
- Controller viết toàn bộ logic
- Không có DI
- Không có 2 implementation riêng
- `sourceImplementation` không khớp
- Không phát hiện inner join làm thiếu dữ liệu
- Không có left join
- Không chuẩn hoá response
- Không có xử lý exception cơ bản

21. Gợi ý tư duy triển khai

Thứ tự làm bài nên như sau:

1. Tạo project Web API bằng .NET 10
2. Tạo entity
3. Tạo seed data bằng list

4. Tạo DTO
 5. Tạo interface `IMaterialService`
 6. Viết `MethodSyntaxMaterialService`
 7. Viết `QuerySyntaxMaterialService`
 8. Đăng ký DI trong `Program.cs`
 9. Tạo controller thật mỏng
 10. Chuẩn hoá response
 11. Xử lý exception not found
 12. Làm thêm left join
 13. Test bằng Swagger/Postman
 14. Tự quan sát dữ liệu bị thiếu khi inner join
-

22. Lưu ý

- Bài này **không yêu cầu database**
- Nhưng vẫn phải làm đúng tư duy **kiến trúc nhiều layer**
- Các layer **không phụ thuộc trực tiếp bừa bãi**, phải đi qua interface và DI
- Sinh viên phải **vibe code nhưng vẫn hiểu mình đang làm gì**
- Ưu tiên:
 - đúng bản chất công nghệ
 - đúng version mới
 - đúng tư duy phân lớp
 - đúng bản chất join
 - đúng DTO
 - đúng chuẩn API